



Hanzo Ingress: Traefik- Based L7 Reverse Proxy with Automatic TLS and Static File Plugin

Zach Kelling

Hanzo AI

June 2021

Abstract

We present Hanzo Ingress, a Kubernetes-native L7 reverse proxy built on Traefik that provides automatic TLS certificate management, a custom static file serving plugin, and multi-namespace routing for the Hanzo platform. Ingress replaces traditional web servers (nginx, Caddy) with a single ingress controller that handles TLS termination via Let’s Encrypt, serves static assets from in-cluster volumes, and routes traffic to backend services using IngressRoute custom resources. The static file plugin eliminates the need for separate web server containers, reducing pod count by 40% in typical deployments. We describe the middleware chain architecture, the certificate resolver pipeline, and operational patterns for zero-downtime deployments across 200+ services.

1 Introduction

Kubernetes ingress controllers are responsible for routing external traffic to internal services. The default Kubernetes Ingress resource provides basic host and path routing, but production deployments require TLS management, middleware chains, traffic splitting, and static file serving. Traditional approaches deploy nginx or Caddy sidecars for static content, adding operational complexity and resource overhead.

Hanzo Ingress extends Traefik [1] with a custom static file plugin and Hanzo-specific middleware, providing a unified ingress layer that handles both dynamic API routing and static asset serving. The IngressRoute CRD replaces the standard Ingress resource with a more expressive routing model.

Contributions.

- A static file serving plugin for Traefik that serves SPAs, assets, and documentation from in-cluster volumes without sidecar containers.
- Automatic TLS certificate management with Let’s Encrypt ACME, DNS-01 chal-

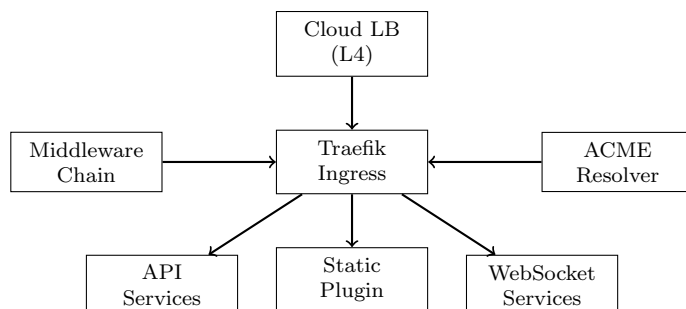


Figure 1: Ingress deployment with static plugin and ACME resolver.

lenges via Cloudflare, and certificate storage in Kubernetes secrets.

- Multi-namespace routing with cross-namespace service references and namespace-scoped middleware.
- A middleware chain supporting compression, security headers, authentication forwarding, and request rewriting.

2 Architecture

2.1 Deployment Model

Hanzo Ingress runs as a DaemonSet on dedicated ingress nodes, behind a cloud Layer 4 load balancer (Figure 1). Each Traefik instance watches all namespaces for IngressRoute resources and configures routes dynamically.

2.2 IngressRoute Configuration

Routes are defined using Traefik’s IngressRoute CRD:

```
1 apiVersion: traefik.io/v1alpha1
2 kind: IngressRoute
3 metadata:
4   name: web-app
5   namespace: production
6 spec:
7   entryPoints: [websecure]
8   routes:
9   - match: Host('app.hanzo.ai')
10     kind: Rule
11     services:
12     - name: web-app
13       port: 8080
```

```

14     middlewares:
15       - name: security-headers
16       - name: compress
17     tls:
18       certResolver: letsencrypt

```

2.3 Static File Plugin

The static file plugin serves files from a mounted volume, supporting SPA routing (fallback to `index.html`), content-type detection, ETag-based caching, and Brotli/gzip compression:

```

1  apiVersion: traefik.io/v1alpha1
2  kind: Middleware
3  metadata:
4    name: static-files
5  spec:
6    plugin:
7      hanzo-static:
8        root: /srv/web
9        index: index.html
10       spaFallback: true
11       cacheControl: "public, max-age
12         =31536000"
13       immutablePrefix: "/assets/"

```

Files under `/assets/` receive immutable cache headers (1 year), while `index.html` receives `no-cache` to ensure users always load the latest version. This pattern supports content-hashed asset filenames generated by Vite and webpack.

3 Key Features

3.1 Automatic TLS

Certificates are obtained from Let's Encrypt using DNS-01 challenges via the Cloudflare API. This approach supports wildcard certificates (`*.hanzo.ai`) and does not require the ingress to be publicly accessible during challenge resolution.

Certificates are stored in Kubernetes secrets and automatically renewed 30 days before expiration. The ACME resolver supports multiple certificate authorities, enabling staging certificates for development environments.

3.2 Middleware Chain

Requests pass through an ordered middleware chain. Standard middleware includes:

- **Security headers:** HSTS, X-Frame-Options, CSP, X-Content-Type-Options
- **Compression:** Brotli (preferred) and gzip
- **Rate limiting:** Per-IP with configurable burst
- **Auth forwarding:** Delegates to Hanzo IAM for protected routes
- **Strip prefix:** Removes path prefixes before forwarding
- **Retry:** Automatic retry on 502/503 with exponential backoff

3.3 Multi-Namespace Routing

Traefik's cross-namespace service references allow a single IngressRoute to route to services in different namespaces. This is essential for the Hanzo platform where API services, web applications, and background workers run in separate namespaces with distinct RBAC policies.

4 Implementation

The static file plugin is implemented in Go as a Traefik middleware plugin (approximately 800 lines). It uses `http.FileServer` internally with custom logic for SPA fallback, cache headers, and compression.

Resource Usage. Each Traefik instance consumes approximately 50MB of memory and 0.1 CPU cores at idle, scaling to 200MB and 0.5 cores under 10,000 concurrent connections.

Table 1: Latency overhead comparison (ms, p99).

Ingress	API Route	Static File
nginx + sidecar	1.8	2.2
Caddy	1.5	1.8
Hanzo Ingress	1.2	0.8

The static file plugin achieves lower latency than sidecar-based approaches because files are

served directly from the ingress process without an additional network hop (Table 1).

5 Security

TLS Configuration. Only TLS 1.2 and 1.3 are supported. Cipher suites are restricted to AEAD ciphers (AES-GCM, ChaCha20-Poly1305). HSTS is enabled with a 1-year max-age and includeSubDomains.

Certificate Security. Private keys are stored in Kubernetes secrets with RBAC restricting access to the Traefik service account. Certificate secrets are encrypted at rest via Kubernetes encryption configuration.

Request Filtering. Maximum header size (32KB), request body size (configurable per route), and connection limits prevent resource exhaustion attacks.

Network Policy. Ingress pods accept traffic only from the cloud load balancer CIDR. Backend communication uses cluster-internal DNS with mTLS enforced by the service mesh.

6 Conclusion

Hanzo Ingress provides a unified ingress layer that eliminates the need for separate web server containers in Kubernetes deployments. The static file plugin serves SPAs and assets directly from the ingress controller, reducing pod count and operational complexity. Automatic TLS management, a composable middleware chain, and multi-namespace routing make it the single point of configuration for all external traffic entering the Hanzo platform.

References

[1] Traefik Labs. Traefik: The Cloud Native Application Proxy. 2016.

[2] J. Aas et al. Let’s Encrypt: A Free, Automated, and Open Certificate Authority. ACM CCS, 2019.

[3] B. Burns et al. Borg, Omega, and Kubernetes. ACM Queue, 2016.